Hybrid NLQ for Financial Data: Combining Vector Search and SQL Generation on BirdSQL

Fatih Ay
METU
Ankara, Turkiye
e273915@metu.edu.tr

Muhammet Kasım Pamuk
METU
Ankara, Turkiye
kasim.pamuk@metu.edu.tr

Abstract

Translating Natural Language Queries (NLQ) into SQL is a challenging task, particularly when dealing with domain-specific semantic ambiguity and fuzzy matching requirements in financial datasets. Traditional "Pure SQL" generation approaches often fail when users employ vague terms (e.g., "young", "popular") or misspelled entities, while "Pure Vector" search approaches struggle with complex numeric filtering and structural joins. In this paper, we present a Hybrid NLQ approach applied to the BirdSQL Financial Domain dataset. Our system utilizes a two-stage pipeline: first, a fine-tuned DistilBERT classifier routes user input to filter out non-database intents, optimizing computational resources. Second, valid queries are decomposed into a vector search plan for semantic retrieval and a structural plan for SQL filtering via LLM. By injecting vector search results (e.g., specific district names) into dynamically generated SQL queries, we effectively bridge the gap between fuzzy intent and structured database execution. We demonstrate that this hybrid architecture achieves high classification accuracy (Accuracy: 0.574) and improved retrieval performance on complex financial queries. **Implementation:** The code and supplementary materials (e.g., data, documentation) are available at:
https://github.com/b21945815/514-code

CCS Concepts

• **Information systems** → **Question answering**; **Information extraction**; • **Computing methodologies** → **Semantic parsing**.

Keywords

Text-to-SQL, Vector Search, Information Extraction, Large Language Models, BirdSQL

ACM Reference Format:
Fatih Ay and Muhammet Kasım Pamuk. 2026. Hybrid NLQ for Financial Data: Combining Vector Search and SQL Generation on BirdSQL. In *CENG 514: Data Mining Project Report*. ACM, New York, NY, USA, 5 pages.

1 Introduction

The ability to query relational databases using natural language is a long-standing goal in data mining and information retrieval. However, real-world datasets present unique challenges. Users often formulate queries containing both exact constraints (e.g., numerical filters or date ranges) and fuzzy, semantic terms (e.g., descriptive adjectives or ambiguous entity names). This made a vocabulary mismatch problem where the semantic intent of the user does not map directly to the lexical structure of the database schema.

CENG 514, Ankara, Turkiye
2026.

While recent advancements in Large Language Models (LLMs) have significantly improved text generation capabilities, deploying them for direct SQL generation reveals critical limitations.
Recent advancements in this domain generally center around two primary perspectives: Prompt Engineering and Retrieval-Augmented Generation (RAG). Prompt engineering focuses on optimizing schema presentation and "Chain-of-Thought" reasoning to guide LLMs toward syntactically correct SQL. Conversely, RAG-based approaches aim to bridge the gap between user vocabulary and database values by retrieving relevant context before generation.
Despite these advancements, general Text-to-SQL systems face several pressing challenges:
1. Semantic Detachment in Pure SQL Approaches For schema linking, existing LLM-based methods typically rely on exact keyword matching or the model's internal training data. However, relational schemas are often designed with rigid, normalized codes that lack semantic context. Traditional "Pure SQL" generators struggle when user terminology is abstract or in natural language. Without an external semantic lookup mechanism, LLMs frequently hallucinate invalid column values or fail to map vague concepts to specific database constraints, leading to executable but logically incorrect queries.
2. Structural Inadequacy of Pure Vector Search While vector databases excel at semantic similarity matching and handling unstructured text, they exhibit significant shortcomings in handling structured relational logic. Standard SQL queries often require precise numeric filtering (e.g., inequality operators like >, <), temporal arithmetic, or complex multi-table joins. "Pure Vector" approaches flatten these structural relationships into similarity scores, losing the ability to perform the mathematical and logical operations required for accurate database retrieval.
To address this, we propose a **Hybrid Approach** that combines the strengths of both methods. Our system decomposes the user query via LLM, utilizing vector embeddings to resolve semantic ambiguities and retrieving the values, which are then injected into a structured SQL query for final execution. We validate this approach using the BirdSQL Financial Domain schema.

2 Related Work

The domain of Text-to-SQL has transitioned from a problem of syntactic parsing to one of complex semantic reasoning. While early benchmarks like Spider encouraged models to master structural composition within static boundaries, the introduction of the BIRD benchmark exposed the fragility of these systems in real-world, "dirty" database environments. Recent advancements now focus on three critical architectural paradigms: Agentic Decomposition, Hybrid Retrieval, and Dynamic Interaction.

2.1 Agentic Decomposition and Reasoning

As query complexity increases "one-shot" prompting often fails due to reasoning drift. To combat this, **DIN-SQL** [3] introduced a decomposed in-context learning framework. By breaking the task into schema linking, query classification, and stepwise generation, DIN-SQL demonstrated that structured reasoning outperforms simple model scaling.Building on this, **MAC-SQL** [4] formalized decomposition into a multi-agent system comprising specialized agents for "Decomposition," "Selection," and "Refinement." They are solving the queries dynamical iteratively, significantly reducing structural hallucinations in complex schemas.

2.2 Hybrid Architectures: Text2VectorSQL

Traditional Text-to-SQL systems fail when users employ semantic filters (e.g., "vintage products") that do not map to exact database keywords. A new class of **Hybrid Architectures**, formally defined by the **Text2VectorSQL** framework [5], addresses this by fusing relational logic with vector search. Some part of it constructs the relational skeleton (joins, numeric filters), while the other parts retrieves entity IDs via embedding similarity. This approach effectively bridges the "vocabulary mismatch" problem, allowing for queries that require both rigid mathematical constraints and fuzzy semantic matching—a critical requirement for our proposed system.

2.3 From Static Generation to Dynamic Interaction

The static nature of traditional benchmarks fails to capture the iterative nature of real-world data analysis. The **BIRD-INTERACT** benchmark [2] highlights a drastic "interaction gap," showing that flagship models like GPT-4 struggle when required to clarification for ambiguous queries. While interactive frameworks like **Interactive-Text-to-SQL** [6] achieve high accuracy by treating SQL generation as a conversational debugging process, they introduce significant latency and cost. Our work differentiates itself by adopting a **Single-Turn Hybrid Strategy**. Instead of relying on costly multi-turn dialogues to resolve ambiguity, we leverage pre-execution vector injection and intent classification to resolve semantic uncertainties upfront, optimizing for both accuracy and computational efficiency.

3 Methodology

3.1 Dataset Schema

We utilize the BirdSQL Financial Domain schema, consisting of interconnected tables including `client`, `district`, `account`, `card`, and `loan`. Key challenges in this schema include:

- **District Names:** Stored in column `A2` (e.g., "Praha", "Brno") requiring text matching
- **Derived Semantics:** Concepts like "young" must be derived from `birth_date`.
- **Categorical Data:** `card.type` ("gold", "classic").
- **Multi Language Data:** `trans.k`$_{symbol}$("POJISTNE"*etc.*)

3.2 System Architecture

Our pipeline consists of distinct steps designed to minimize "token waste" and maximize retrieval accuracy.

3.2.1 Phase 1: Context-Aware Intent Classification. Before invoking the computationally expensive decomposition agent, the system must distinguish between conversational input (General Chat) and analytical requests (Database Queries). This routing step prevents the system from attempting to generate SQL for inputs such as "Hello" or "How are you?". We fine-tuned a **DistilBERT** model for this binary classification task. DistilBERT was selected over larger generative models due to its low latency and high inference speed, which are critical for real-time applications. The model classifies inputs into two categories:

- **Label 0 (General Chat):** Conversational fillers, greetings, and non-technical questions.
- **Label 1 (Database Query):** Questions requesting data retrieval, aggregation, or filtering (e.g., "Show me sales data").

Only inputs classified as "Database Query" with a high confidence score proceed to the query decomposition phase.

3.2.2 Phase 2: Schema-Guided Query Decomposition. To address the non-deterministic nature of LLM outputs, we moved away from direct Text-to-SQL generation. Instead, we implemented a **Schema-Guided Decomposition** phase where the LLM functions as a "SQL Decomposition Expert" The core of this phase is a recursive JSON-based Intermediate Representation (IR) defined by a strict prompt template. This IR decouples the *structure* of the query from the *values* within it. **The Intermediate Representation (IR):** The decomposition module constructs a JSON object consisting of independent "Tasks." Each task represents a query unit defined by the following grammar:

- **ValueNode:** The atomic unit of data, categorized into types such as `LITERAL` (numbers, dates), `COLUMN` (schema references), and `FUNCTION` (aggregations).
- **ConditionNode:** A logic tree representing filtering rules (Recursive `LEAF` and `BRANCH` nodes) used in `WHERE` and `HAVING` clauses.
- **StructuralLogic:** Explicit definitions for relational algebra operations, specifically `JOIN` conditions and Set Operations (`UNION`, `INTERSECT`).

Aggressive Semantic Tagging: A critical innovation in our prompt engineering is the "Aggressive Semantic Search" rule. The LLM is strictly instructed *not* to assume exact string matching for text fields. Instead, it must generate a `SEMANTIC` node:

```
{ "type": "SEMANTIC", "value": "rich",
  "column": "description", "table": "client" }
```

This instruction ensures that fuzzy terms (e.g., "rich", "Prague") are isolated for the subsequent Vector Search phase, while rigid constraints (dates, IDs) remain as `LITERAL` nodes.

Logic Validation: The decomposition template also enforces correct arithmetic syntax for SQLite, specifically addressing integer division pitfalls by enforcing `CASE WHEN` logic for ratios and percentages (e.g., casting counts to 1.0 before division).

We implemented this decomposition engine using two distinct backends to test generalization: `GPT-4o` for high-precision reasoning and `Groq (hosting gpt-oss-120b)` for high-speed inference. The prompt structure remains consistent across models, ensuring the resulting JSON is parser-friendly.

1. ValueNode: The atomic unit of data.
- LITERAL: `{"type": "LITERAL", "value": ...}`
- COLUMN: `{"type": "COLUMN", "value": "t.col"}`
- FUNCTION: `{"type": "FUNCTION", "name": "SUM", ...}`
- CONDITION: `{ "type": "CONDITION", "value": ConditionNode }`
- SEMANTIC: `{"type": "SEMANTIC", "value": "rich", ...}`

2. ConditionNode: Logic tree for filtering.
- LEAF: `{"left": ValueNode, "op": "=", "right": ...}`
- BRANCH: `{"logic": "AND|OR", "conditions": [...]}`

3. StructuralLogic: Relational operations.
- JOIN: `{"type": "INNER JOIN", "on": ConditionNode}`
- SET_OP: `{"type": "UNION|INTERSECT", "task_id": 3}`

4. SelectNode: Target list items.
- `{"value": ValueNode, "alias": "col_alias"}`

5. OrderNode: Sorting rules.
- `{"value": ValueNode, "direction": "ASC|DESC"}`

Figure 1: Example of the Semantic Schema Structure used for Query Decomposition in Prompt

Schema Representation Strategy: Instead of raw DDL (Data Definition Language) statements, we inject a semantic-rich JSON schema into the model's context window. This JSON structure encapsulates three critical layers:

Structural Metadata: Table names, column types, and foreign key relationships to ensure valid SQL generation.

Semantic Annotations: Natural language descriptions for cryptic column / data names (e.g., mapping district.A3 to "region format") which are meaningless without domain knowledge.

3.2.3 Phase 3: Cross-Lingual Vector Retrieval. A critical challenge in the BirdSQL dataset is the linguistic disparity between user queries (English) and database content (Czech). For example, a user might query for "Prague" or "insurance payment", while the database stores these as "Hl.m. Praha" and "POJISTNE", respectively. Standard English-trained models failed to align these disparate tokens effectively.

To address this, we implemented a **Cross-Lingual Retrieval Module** using the `paraphrase-multilingual-MiniLM-L12-v2` model. This model projects English and Czech terms into a shared semantic vector space. We utilized a **Dual Indexing Strategy** to handle different data types:

- **Direct Indexing (Raw Values):** For named entities like cities or regions (e.g., `district.A2`), we embed the distinct raw values directly. The multilingual model is capable of aligning the English vector for "Prague" with the Czech vector for "Hl.m. Praha" due to its cross-lingual pre-training.
- **Mapped Indexing (Semantic Translation):** For cryptic internal codes (e.g., `trans.k_symbol` or `loan.status`), raw value embedding is insufficient (e.g., embedding the string "B" is meaningless). We employ a metadata mapping layer where we embed a descriptive English definition (e.g., "contract finished, loan not paid") but store the original database code (e.g., "B") as the retrieval payload.

During execution, the system performs a similarity search in the column-specific collection (e.g., `district_A2`) and calculates a confidence score ($S = \frac{1}{1+d}$) based on the L2 distance d. If a match exceeds a set threshold, the retrieved database value is returned for SQL injection.

3.2.4 Phase 4: Hybrid SQL Generation. The final SQL query is constructed by injecting the retrieved IDs into the `WHERE` clause. Instead of forcing the LLM to hallucinate district names, we generate: `WHERE district.A2 = "Praha - Zapad"`. This is combined with standard SQL filters (e.g., `AND loan.amount > 100000`) to ensure structural correctness.

4 Experimental Setup

4.1 Intent Classifier Training

The Intent Classification model was fine-tuned using the distilbert-base-uncased architecture. The training dataset was constructed by combining:

(1) **General Chat:** 50 samples of common conversational phrases (e.g., "Tell me a joke", "Good morning").
(2) **BirdSQL Queries:** 50 samples extracted from the BirdSQL training set to represent valid financial queries.

The model was evaluated on a held-out test set containing a mix of new general chat phrases and unseen BirdSQL development queries.

4.2 LLM Configuration

To ensure deterministic output during the Query Decomposition phase, we configured both the OpenAI (`gpt-4o`) and Groq (`gpt-oss-120b`) engines. Furthermore, we enforced structural compliance by enabling the `json_object` response format mode[cite: 2], which constrains the model to generate valid JSON syntax. This significantly reduced parsing errors compared to free-text generation.

4.3 Hybrid Pipeline Setup

- **Vector Store:** We utilized **ChromaDB** as the persistent vector store due to its efficient handling of separate collections for each schema column.
- **Embeddings:** We replaced the standard `all-MiniLM-L6-v2` with the **paraphrase-multilingual-MiniLM-L12-v2** model. This upgrade was necessary to handle the mixed-language nature of the dataset (English Queries → Czech Data) and resulted in a significant improvement in retrieving localized entities.
- **Indexed Columns:** To maximize semantic coverage, we created dedicated vector indices for 17 distinct schema attributes across four categories:
  - *District Attributes:* `district.A2` (Name), `district.A3` (Region), and demographic indicators (`district.A4` – `district.A7`).
  - *Transaction & Order Details:* `trans.type`, `trans.operation`, `trans.k_symbol`, `trans.bank`, `order.bank_to`, `order.k_symbol`.
  - *Entity Classification:* `account.frequency`, `card.type`, `client.gender`, `disp.type`, `loan.status`.

Table 1: Vector Search Retrieval Examples (English Query → Czech DB Value)

User Query Term	Target Column	Retrieved DB Value
"Prague"	district.A2	Hl.m. Praha
"lower Moravia"	district.A3	south Moravia
"pension"	trans.k_symbol	DUCHOD
"insurenje" (Typo)	trans.k_symbol	POJISTNE
"NOT paid yet"	loan.status	B

5 Results

5.1 Intent Classification Performance

The fine-tuned DistilBERT router demonstrated high reliability in filtering non-SQL traffic. On the test set of 46 samples, the model achieved:

- **Accuracy:** 97.83%
- **F1-Score:** 0.9787

The model successfully differentiated between semantically similar but functionally different sentences. For example, "How many items did we sell?" was correctly classified as a Query (Conf: 98.7%), while "I feel lonely" was classified as Chat (Conf: 98.7%).

We observed a specific edge case in the query *"Who won the World Cup?"*. The model classified this as a **Database Query** (Label 1) with 96.9% confidence. This indicates that the model has learned to associate "Who won..." interrogatives with data retrieval, even if the specific domain (Sports) is not in the financial database. This suggests future work may require domain-specific entity filtering alongside intent classification.

5.2 End-to-End Evaluation

We evaluated our Hybrid NLQ approach against two state-of-the-art baselines: **DIN-SQL** [3] and **DAIL-SQL** [1]. To ensure a fair comparison, both baseline methodologies were manually re-executed on the BirdSQL Financial Database using the `GPT-4o` engine.

Table 2 presents the comparative results across accuracy and computational cost metrics. The success metrics are defined as follows based on the result set comparison

- **Exact Match:** The result set is completely identical, including both row content and ordering.
- **Strict SQL:** The result content is identical, but the row ordering differs (e.g., missing or different `ORDER BY` clause).
- **Soft Retrieval:** The rows are identical, but the column ordering in the `SELECT` clause differs.

5.2.1 Accuracy vs. Efficiency Trade-off. Our Hybrid (GPT-4o) approach achieved a success rate of **57.40%** without hints, outperforming the DAIL-SQL baseline (48.14%) significantly. While DIN-SQL achieved the highest accuracy (71.30%), it comes at a prohibitive computational cost. As shown in the "Average Token" column, DIN-SQL consumes approximately **21,622 tokens** per query due to its multi-step reasoning and self-correction modules.

In contrast, our Hybrid approach requires only **3,017 tokens** on average—a **7x reduction** in cost. This demonstrates that injecting vector search results directly into the SQL generation process allows for competitive accuracy with a fraction of the token budget, making it a more viable solution for low-latency, real-time applications.

5.3 Error Analysis

To better understand the limitations of our system, we conducted a granular error analysis on the 46 failed queries from the Hybrid (GPT-4o) run. The distribution of error categories is shown in Table 3. Note that some queries exhibited multiple types of errors simultaneously.

1. Logical Inference (42.9%): The primary bottleneck was not retrieval, but complex reasoning. The model occasionally struggled with multi-hop reasoning (e.g., nesting aggregations correctly) or misinterpreting domain-specific financial logic

2. Validity of Ground Truth (18.4%): A significant portion of "failures" were attributed to ambiguities in the BirdSQL benchmark itself. In several cases, our system generated a logically valid SQL query that returned a correct answer, but the benchmark's "Gold SQL" used a different, sometimes debatable, sometimes wrong interpretation of the natural language question.

3. Efficacy of Vector Search: Notably, "Value Retrieval & Schema Linking" errors accounted for only **14.3%** of failures. This low error rate validates our core hypothesis: offloading entity linking to a dedicated Vector Search module is more reliable than relying solely on the LLM's parametric knowledge for fuzzy matching.

5.4 Qualitative Analysis: Case Study

To illustrate the efficacy of the hybrid pipeline, we present a detailed breakdown of a complex query requiring multi-hop joins, fuzzy entity matching, and temporal reasoning.

User Query:

> *"Show me young female clients from Prague who have gold cards and loan amounts over 100,000"*

Generated SQL:

```
SELECT c.client_id, c.gender, d.A2, cr.type, l.amount
FROM "client" as c
INNER JOIN "district" as d
   ON (c.district_id = d.district_id)
INNER JOIN "disp" as p
   ON (p.client_id = c.client_id)
INNER JOIN "card" as cr
   ON (cr.disp_id = p.disp_id)
INNER JOIN "loan" as l
   ON (l.account_id = p.account_id)
WHERE c.gender = 'F'
  AND d.A2 = 'Praha - zapad'
  AND cr.type = 'gold'
  AND l.amount > 100000
  AND c.birth_date > '1995-01-01'
```

Analysis of Success Factors: This result highlights three critical capabilities of our system:

(1) **Cross-Lingual Entity Injection:** The user requested "Prague" (English). The Vector Search module successfully retrieved the localized Czech entity `'Praha - zapad'` (Prague West)

Table 2: Overall Performance Comparison on BirdSQL Financial Domain. The "Hint" configuration includes evidence from BIRD dataset. (*Token counts for DIN-SQL and DAIL-SQL are estimated based on original paper reports, as they were not instrumented in this experiment.)

Model Configuration	Success	Acc (%)	Exact SQL	Strict SQL	Soft Ret.	Avg Tokens
Hybrid (GPT-4o)	**62 / 108**	**57.40%**	49	0	2	**3,017**
Hybrid (Groq/Llama3-70b)	58 / 108	53.70%	43	0	2	3,789
Hybrid (GPT-4o) + Hint	67 / 108	62.00%	49	0	3	3,062
Hybrid (Groq) + Hint	65 / 108	60.20%	51	1	1	4,034
DIN-SQL (GPT-4o + Hint)	77 / 108	71.30%	67	0	1	21,622*
DAIL-SQL (GPT-4o + Hint)	52 / 108	48.14%	40	1	11	~6,000*

Table 3: Error Distribution for Hybrid (GPT-4o) Configuration.

Error Category	Count	Share (%)
General Logical Inference Failures	21	42.9%
Ground Truth & Evaluation Validity	9	18.4%
Output Format Compliance	8	16.3%
Value Retrieval & Schema Linking	7	14.3%
Missing Info in Prompt	4	8.1%

and injected it directly into the `WHERE` clause, resolving the vocabulary mismatch.

(2) **Temporal Reasoning:** The abstract term "young" was interpreted by the Query Decomposition agent as a specific date filter (`birth_date > '1995-01-01'`), demonstrating the LLM's ability to ground semantic adjectives into concrete database logic.

(3) **Complex Schema Navigation:** The system correctly identified the optimal join path across five tables (`client` → `district` → `disp` → `card` → `loan`) to link clients to their loans and cards, a structural task where pure vector approaches typically fail.

6 Conclusion

In this paper, we presented a Hybrid NLQ system for the financial domain that effectively bridges the gap between semantic user intent and database schemas. By separating semantic retrieval (via Vector Search) from structural logic, we addressed the "vocabulary mismatch" problem inherent in financial datasets.

Our experimental results on the BirdSQL dataset demonstrate that this hybrid architecture achieves competitive accuracy (57.4% without hints) while maintaining superior computational efficiency, consuming 7x fewer tokens than leading decomposition-based methods like DIN-SQL. Future work can be on improving the logical inference capabilities of the decomposition agent, exploring multi-turn correction strategies to further reduce reasoning errors and obtaining database info dynamically.

Acknowledgments

We thank the course instructor of CENG514, Pınar Karagöz for her guidance.

References

[1] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. (2023). arXiv:2308.15363 [cs.DB] https://arxiv.org/abs/2308.15363

[2] Nan Huo, Xiaohan Xu, Jinyang Li, Per Jacobsson, Shipei Lin, Bowen Qin, Binyuan Hui, Xiaolong Li, Ge Qu, Shuzheng Si, Linheng Han, Edward Alexander, Xintong Zhu, Rui Qin, Ruihan Yu, Yiyao Jin, Feige Zhou, Weihao Zhong, Yun Chen, Hongyu Liu, Chenhao Ma, Fatma Ozcan, Yannis Papakonstantinou, and Reynold Cheng. 2025. BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation for Large Language Models via Lens of Dynamic Interactions. (2025). arXiv:2510.05318 [cs.AI] https://arxiv.org/abs/2510.05318

[3] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. arXiv:2304.11015 [cs.CL] https://arxiv.org/abs/2304.11015

[4] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, LinZheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. (2025). arXiv:2312.11242 [cs.CL] https://arxiv.org/abs/2312.11242

[5] Zhengren Wang, Dongwen Yao, Bozhou Li, Dongsheng Ma, Bo Li, Zhiyu Li, Feiyu Xiong, Bin Cui, Linpeng Tang, and Wentao Zhang. 2025. Text2VectorSQL: Towards a Unified Interface for Vector Search and SQL Queries. (2025). arXiv:2506.23071 [cs.CL] https://arxiv.org/abs/2506.23071

[6] Guanming Xiong, Junwei Bao, Hongfei Jiang, Yang Song, and Wen Zhao. 2025. Multi-Turn Interactions for Text-to-SQL with Large Language Models. (Nov. 2025), 3560–3570. doi:10.1145/3746252.3761052